Department of Information Science & Engineering

Experiential Learning (EL)

SDG 9 – Industry, Innovation & Infrastructure

PROJECT SYNOPSIS

Phase 1

Title of Project:	ChipCheck: AI-Driven Adaptive Physical Testing for Embedded Systems (Adaptive Hardware Fuzzing Testbench)
Team No.:	29
Student Name(s):	Kavin Krishnan C Chinmai Kiran Shetti Sumukha Upadhyaya Snehal Reddy Thadigotla Sutej R Haritsa
USN / Reg. No.:	1RV24CS126 1RV24CS068 1RV24IS130 1RV24IS126 1RV24CY055
Guide:	Dr. H K Krishappa
Semester and Section:	Vth Semester [section to be filled]
Date of Submission:	12 / 11 / 2026

Academic Year: 2026–27

Table of Contents

1. Abstract 1
2. Introduction 2
3. Literature Survey / Review of Related Work 3
4. Objectives 5
5. Proposed Methodology 6
6. Expected Outcomes & Deliverables 16
7. Evaluation Metrics & Validation Plan 17
8. References 20
9. Declaration & Signatures 22

List of Abbreviations

AHFT	Adaptive Hardware Fuzzing Testbench
DUT	Device Under Test
HIL	Hardware-in-the-Loop
SDG	Sustainable Development Goal
MCU	Microcontroller Unit
ADC	Analog-to-Digital Converter
GPIO	General-Purpose Input/Output
LDR	Light-Dependent Resistor (photoresistor)
JSONL	JSON Lines (one JSON object per line)
REST / SSE	Representational State Transfer / Server-Sent Events
TPE	Tree-structured Parzen Estimator
GP-BO	Gaussian-Process Bayesian Optimisation
ET	ExtraTrees (extremely randomised trees)
PRD	Product Requirements Document
HLD	High-Level Design (document)
RQ	Research Question

1. Abstract

Embedded products fail under particular combinations of operating conditions such as supply voltage, workload, peripheral state and event timing. These combinations are rare among all possible ones, and testing every one of them on real hardware is too slow. Random testing wastes experiments in regions where the device already behaves correctly. This project builds ChipCheck, a safety-constrained Adaptive Hardware Fuzzing Testbench (AHFT) that uses the results of earlier physical experiments to choose the next one.

An engineer writes a test contract that lists the controllable variables, measurable outputs, legal limits, hard safety limits and the failure requirement. A planner proposes a candidate experiment. A deterministic safety check accepts or rejects it in the backend and again in the bench controller. The controller applies the experiment to the device, collects synchronised measurements and returns the setup to a safe state. An independent oracle then labels the result.

The first deliverable targets timing failures on an ESP32. A laptop sends a six-parameter experiment (sampling interval, duration, sensor, task load, deadline and warm-up) over USB as JSON Lines. The board returns a microsecond timestamp record for every task release. The laptop recomputes every statistic from the raw samples and applies one frozen deadline rule, so the firmware's own summary is only advisory. Adaptive selection uses an ExtraTrees surrogate whose acquisition combines failure probability, model uncertainty and novelty, and it is compared with random, grid, TPE and space-filling baselines at equal experiment budgets. Failures are confirmed and minimised. The software is built simulator-first, with disclosed seeded timing defects, so that connecting the real board changes one adapter. The expected outcome is a reproducible testbench and honest evidence on whether adaptive search finds failures in fewer physical experiments, without treating the absence of a failure as proof of safety.

Keywords: *Adaptive Testing, Hardware-in-the-Loop, Embedded Systems, Timing Failures, Fuzzing, Surrogate Models, ExtraTrees, Safety-Constrained Search, ESP32*

2. Introduction

2.1 Background

This project is aligned with UN Sustainable Development Goal 9 (Industry, Innovation and Infrastructure), Target 9.5, which supports technological research and the upgrading of industrial technologies [1]. Cheaper and more systematic validation of embedded hardware is a concrete step toward it.

Embedded systems sit inside cars, medical devices, factory equipment and consumer electronics, and the cost of a field failure is high. Their failures are often not caused by one wrong input. They appear when several conditions coincide: a supply rail near its lower limit while the processor is busy and a peripheral is mid-transfer, or a short sampling period combined with heavy computation. A firmware build that looks correct at nominal settings can still miss deadlines, drift in timing or reset under such a combination.

Software testing has a mature answer to a large input space. Fuzzers keep inputs that reach new behaviour and mutate them, instead of enumerating everything [2]. Surveys of embedded fuzzing show that the idea has been carried to firmware, but also that crash detection is harder on embedded targets and that tools are hard to compare [3],[4]. Most of that work stays in emulation or on the bytes delivered to the firmware. It does not vary the electrical and timing conditions of a real device, which is where many embedded faults live.

Hardware experiments are also expensive. One physical run takes seconds to minutes, has measurement noise, and can wear or stress equipment. A search for failures therefore has to spend a small budget of experiments well and must never leave the limits that keep the board and the bench safe.

2.2 Problem Statement

Conventional embedded testing either sweeps a grid or samples at random, and neither learns from the experiments already run. Adaptive search methods exist, but they are mostly demonstrated in simulation or in software, and they do not combine a reviewed safety envelope, trustworthy physical measurement and failure minimisation in one test loop. There is no small, reproducible testbench that lets a team ask whether feedback-driven selection finds embedded failures in fewer physical experiments than conventional selection, without running anything outside the permitted limits.

2.3 Motivation

A team that spends its budget on settings that are known to pass learns nothing, and a team that spends it beyond the limits may damage the device. A testbench that chooses

informative experiments, refuses unsafe ones in two independent places, and stores enough evidence to replay and shrink every failure would make validation faster and safer. A reproducible open design also lets others test planning methods on real hardware under identical rules, which supports the research and innovation aims of SDG 9.

3. Literature Survey / Review of Related Work

3.1 Fuzzing and Firmware Testing

American Fuzzy Lop popularised a feedback loop that keeps inputs producing new behaviour and mutates promising seeds rather than relying on blind generation [2]. The transferable idea is feedback-guided allocation of test effort. Yun et al. [3] survey fuzzing of embedded and IoT firmware, covering emulation, multi-architecture support and why crash detection is harder in embedded targets, and Eisele et al. [4] classify embedded fuzzers by how they interface with the target and note the lack of standard cross-tool benchmarks. Rehosting tools replace hardware abstraction layers (HALucinator [5]), model peripheral interfaces automatically (P^2IM [6]) or model memory-mapped input/output precisely (Fuzzware [7]) to fuzz firmware in an emulator. μAFL [8] connects a host fuzzer to a real microcontroller and uses processor trace features for feedback, and E-FuzzEdge [9] reduces communication overhead in in-place fuzzing. These systems fuzz the bytes that reach the firmware. None of them varies the electrical conditions or workload timing of the device, and emulation cannot reproduce power integrity, component tolerance or unit-to-unit variation.

3.2 Combinatorial and Space-Filling Test Design

Combinatorial testing covers all chosen low-order interactions of discrete values with far fewer tests than an exhaustive sweep [10]. It is a strong baseline for categorical states, but it is not adaptive and needs discretisation, so it can miss a narrow failing region between two chosen levels of a continuous variable. Latin hypercube sampling stratifies each variable's range and gives broad initial coverage [11]. In this project both are used as baselines or warm starts, not as the contribution.

3.3 Active Learning and Safe Optimisation

Bryan et al. [12] select experiments to locate function threshold boundaries with entropy and variance criteria, and Gotovos et al. [13] estimate level sets with Gaussian processes and confidence-bound sampling. They motivate the goal of learning a pass/fail boundary from few samples. SafeOpt [14] explores an unknown function while staying above a safety threshold under a Gaussian-process assumption, and Berkenkamp et al. [15] extend it to

several safety constraints and apply it to robot parameter tuning. Their assumptions matter: a physical response surface can be discontinuous, stochastic and categorical, and a smooth model can be confidently wrong. In this project the learned model only guides the choice among candidates. The hard limits are checked by deterministic rules that the model cannot override.

3.4 Falsification, Hardware-in-the-Loop and Adaptive Fault Injection

Temporal-logic falsification tools search for inputs that give the lowest robustness value for a requirement, and a negative robustness is a counterexample [16]. Adaptive stress testing frames rare-failure discovery as a sequential search [17]. Hardware-in-the-loop testing is established engineering: PHiLIP offers an external reference device for automated multi-platform embedded-OS testing [18], and IEEE 2004-2025 gives a recommended practice for HIL simulation-based testing [19]. The closest recent precedent is GlitchLab, an unreviewed preprint that adaptively optimises physical voltage and timing fault injection for security faults [20]. It deliberately perturbs execution beyond normal operation, whereas this project stays inside the product's approved non-destructive envelope and targets product requirements.

3.5 Surrogate Models, Minimisation and Benchmarking

The Tree-structured Parzen Estimator models good and bad configurations with two densities and proposes points that maximise their ratio [21]. Extremely randomised trees build an ensemble with random split features and thresholds, which is cheap to train on small data sets, and the spread of the trees gives a heuristic uncertainty [22]. The Optuna framework provides TPE and the random samplers used here as adaptive and baseline planners [23]. Delta debugging simplifies a failure-inducing input to a minimal one [24]; this project applies the same idea to physical experiments. FuzzBench argues that fuzzer claims need repeated trials, shared benchmarks and raw data [25], which this project adopts as a requirement.

3.6 Research Gap Summary

Table 3.1: Research gap summary

Reference	Key Contribution	Limitation / Research Gap
Yun et al., 2023 [3]; Eisele et al., 2022 [4]	Surveys of embedded and IoT fuzzing	Taxonomy only; no standard benchmark and no search over electrical or timing conditions

Clements et al., 2020 [5]; Feng et al., 2020 [6]; Scharnowski et al., 2022 [7]	Firmware rehosting and peripheral modelling for emulated fuzzing	Emulation cannot reproduce power integrity, tolerances or unit variation
Li et al., 2022 [8]; Rusconi et al., 2026 [9]	Feedback fuzzing of real microcontrollers	Input is bytes delivered to firmware; external conditions are not varied
Kuhn et al., 2010 [10]	Combinatorial covering arrays	Not adaptive; needs discretisation of continuous variables
Sui et al., 2015 [14]; Berkenkamp et al., 2023 [15]	Safe exploration with Gaussian processes	Smooth-model assumptions; demonstrated on robots and simulation, not a reviewed embedded test bench
Annapureddy et al., 2011 [16]	Temporal-logic falsification	Needs a simulator model; no electrical safety enforcement or instrument uncertainty
Hossain et al., 2026 [20] (preprint)	Adaptive HIL fault injection	Security-oriented glitching outside the normal envelope; not product requirements
Zeller and Hildebrandt, 2002 [24]	Delta debugging	Defined for software inputs, not stochastic physical episodes

4. Objectives

The primary objectives of this project are:

- To design a safety-constrained test loop in which an engineer-written contract, a planner, a backend safety check, a bench controller and an oracle each have one clear job, and in which no experiment outside the declared limits is executed.
- To build a bench that executes physical experiments on an ESP32 over a typed JSON-Lines protocol and records microsecond-resolution raw samples, with every summary statistic recomputed independently on the PC.
- To implement an adaptive planner (ExtraTrees surrogate with a failure-probability, uncertainty and novelty acquisition) and compare it with random, grid, TPE and space-filling selection at equal experiment budgets.

- To confirm each discovered failure, reduce it to a minimal set of conditions, and publish the exact experiment sequence, raw traces and safety decisions so that every result can be replayed.

4.1 Scope and Delimitations

- *In scope for version 1 (Feature 1):* timing-failure detection on an ESP32 using a software-defined synthetic workload, an LDR or potentiometer as the sensor under test, a deterministic simulator with disclosed seeded defects, and a seven-view operator dashboard.
- *In scope for the wider platform:* the test contract, two-layer safety check, controller and console, and the multi-rail OLED demonstration (programmable supply rails, an ESP32-S3 device and a test-controller MCU) that carries the safety design.
- *Out of scope:* destructive or security fault injection, Wi-Fi and Bluetooth workloads, multiple devices, unattended operation, worst-case-execution-time guarantees, and any claim that a device is proven safe.

5. Proposed Methodology

5.1 System Architecture Overview

The system is split into a control plane on the laptop and a bench plane beside the physical hardware (Figure 5.1). The researcher works in a web console, written in Next.js, whose server routes add the operator token so that the browser never holds it. The backend owns the test contract, the campaign state and the first safety check, and it keeps an append-only audit log in a relational database (SQLite in version 1). The Agent service manages the campaign lifecycle and the experiment budget and asks the Decision service for the next candidate. The Oracle judges measured behaviour. On the bench plane, the Controller claims an approved experiment, re-checks the limits against the live bench state, applies the conditions through the Adapter, collects measurements and returns the setup to a safe state. The Adapter hides the instrument: in simulation it is a deterministic simulated ESP32, and on hardware it is a serial link, so switching is a device-configuration change and not a code change. The Backend and the Controller share one safety-rule library. All application services are written in Python.

Safety is layered so that no single component is trusted alone: the backend checks every candidate against the contract limits, the controller re-checks against the live bench before running, the instruments enforce their own limits, and a watchdog, fuse and E-stop on the multi-rail rig cut power independently of software. A candidate that fails any check is

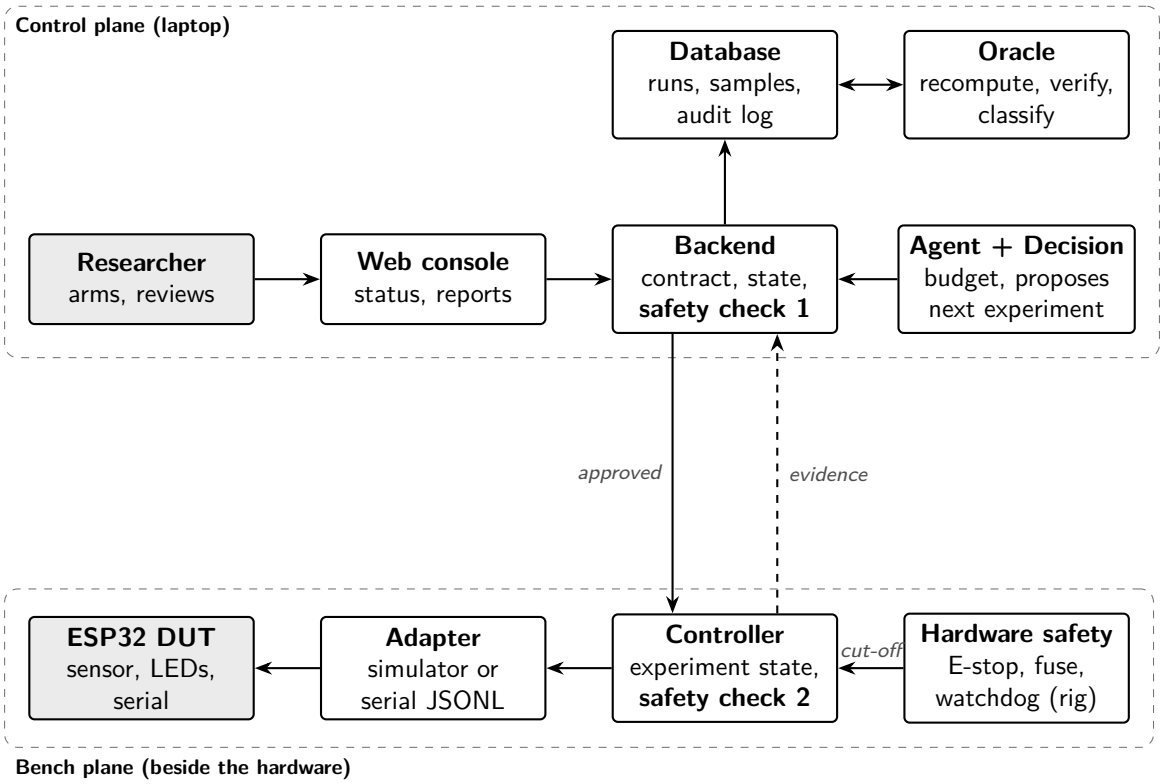


Figure 5.1: Proposed system architecture (control plane and bench plane).

rejected with structured reasons, is never clamped to a nearby legal value, is never executed and is never used for training. The practice follows the principle that hazards in laboratory electrical equipment are controlled by design and not by procedure [26].

5.2 Test Bench Hardware

Version 1 uses an ESP32-WROOM-32 DevKit [26] on a breadboard, connected to the laptop by a micro-USB cable (Figure 5.2). A photoresistor (LDR) is wired to GPIO34 and a trimmer potentiometer to GPIO35, both read by the 12-bit ADC (raw range 0–4095); a third channel, `ADC_SIM`, is synthetic. Figure 5.3 shows a typical 30-pin DevKit of this kind: the module with its PCB antenna under a metal shield, a USB-to-UART bridge (a CP2102 in the model), a 3.3 V regulator (AMS1117), EN and BOOT buttons and a power and a GPIO2 LED. The parts on the physical board will be confirmed by the hardware team. The device runs a synthetic workload whose size, `task_load`, is expressed in calibrated work units so that the load is repeatable across boards. Version 1 changes no voltages and drives no external loads, so it adds no electrical hazard beyond USB-powered bench work. The exact module (WROOM-32 or S3) is to be confirmed by the hardware team and the device reports it truthfully in its capability reply. The multi-rail platform for the OLED demonstration (an ESP32-S3 device, an RP2040 test controller and three programmable extra-low-voltage rails) remains the long-term rig.

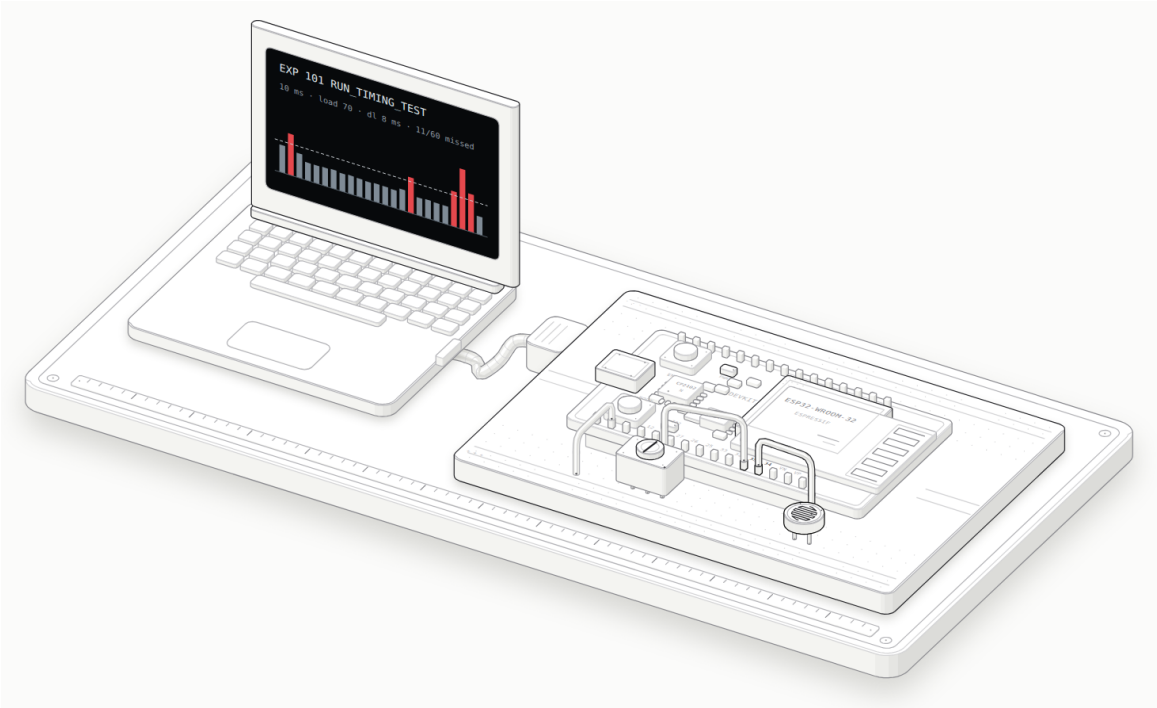


Figure 5.2: The timing bench (isometric model built from the project's bench design). The laptop screen shows, for each task release, how long after its scheduled time the task finished relative to its deadline (dashed line); missed deadlines are the dark bars. The bars come from the same timing model as the project simulator (seeded defect A shown), not from a physical board.

5.3 Experiment Description and Data

The data of this project are experiment records produced by the bench, not a pre-collected data set. An experiment is a six-variable configuration sent from the PC to the ESP32 as one UTF-8 JSON line. The device answers with an acknowledgement, one raw sample frame per scheduled task release and a final summary. Every frame carries a protocol version, and a mismatch is rejected. The capability envelope below is reported by the device and is enforced on both the PC and the device.

Table 5.1: Experiment input variables and capability envelope

Variable	Range	Meaning
`sampling_interval_ms`	5–1000 ms	Period between scheduled task releases
`duration_ms`	100–30000 ms	Length of the measured window
`sensor_type`	LDR, POT, ADC_SIM	Sensor channel read in each task
`task_load`	0–100 units	Calibrated synthetic work per release

`deadline_ms`	2–1000 ms	Relative deadline from each scheduled release
`warmup_ms`	0–5000 ms	Initial window excluded from all statistics

Integers only are accepted for integer fields (no booleans, floats or strings). All problems of a bad request are returned together with codes such as `MISSING_FIELD`, `INVALID_TYPE`, `OUT_OF_RANGE`, `UNSUPPORTED_SENSOR` and `INVALID_WINDOW`, and the device buffer limit of 4096 samples per experiment is a derived constraint. For each release the device records, from one monotonic microsecond clock, the scheduled time, the actual start time, the execution time, the sensor value, a read-success flag, an error code, the boot identifier and a warm-up flag. The PC derives start lateness, completion time and inter-sample interval from these and never trusts the firmware for them. The deadline rule is frozen:

$$\text{a sample misses its deadline if and only if} \quad t_{\text{actual}} + t_{\text{exec}} > t_{\text{scheduled}} + 1000 \cdot d_{\text{ms}}.$$

Both start lateness and execution time therefore count, because the deadline is anchored at the scheduled release. Jitter is the sample standard deviation of the inter-sample intervals, and the deadline miss rate is the number of misses divided by the number of scheduled samples, so samples that never arrived still count against the device.

5.4 Proposed Algorithm / Model

The episode loop (Figure 5.4) is the same for every strategy, so that strategies differ only in how they pick the next experiment.

Step 1: warm start. Until four labelled experiments exist, the planner draws safe candidates that maximise their distance from experiments already run, so that the surrogate is never fitted on a handful of points.

Step 2: surrogate. The planner fits an ensemble of 25 extremely randomised trees [22] on earlier experiments of the same campaign. The inputs are only the six pre-execution variables, never jitter, miss rate, outcome or signature, and the label is the oracle's failure label. A passing experiment that came close to its deadline carries a failure-proximity target between 0 and 0.85, so near-limit passes still inform the model. The mean of the trees is the failure probability $\hat{P}(x)$, and their standard deviation $\hat{U}(x)$ is a heuristic uncertainty.

Step 3: acquisition. Each candidate in a pool of unseen, safe configurations is scored by

$$A(x) = \alpha \hat{P}(\text{fail} \mid x) + \beta \hat{U}(x) + \gamma \widehat{\text{Novelty}}(x),$$

where Novelty is the distance to the nearest experiment already run, each term is min–

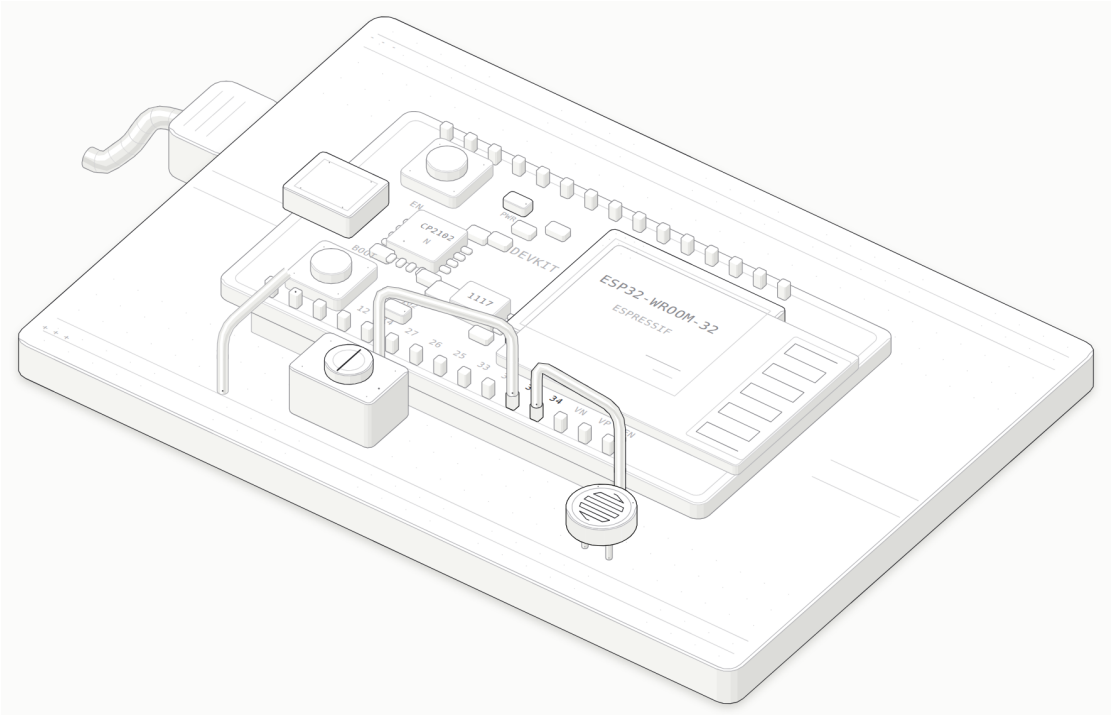


Figure 5.3: Model of a typical ESP32-WROOM-32 DevKit on the breadboard: shielded module with PCB antenna, CP2102 bridge, AMS1117 regulator, EN and BOOT buttons, header pins (GPIO34 and GPIO35 emphasised) and the micro-USB plug. The 3.3 V feed and the pull-down resistor are omitted for clarity.

max normalised over the pool, and the defaults are $\alpha = 0.50$, $\beta = 0.30$, $\gamma = 0.20$. The highest-scoring candidate is proposed, and every selection stores its score breakdown and a readable reason. Ablation presets remove the uncertainty term, the novelty term or the failure-probability term in turn, to test which term matters.

Step 4: safety, execution and labelling. The candidate passes the two safety checks, runs on the device, and the oracle recomputes all statistics from the raw samples, compares them with the firmware summary and labels the result in the order shown in Table 5.2.

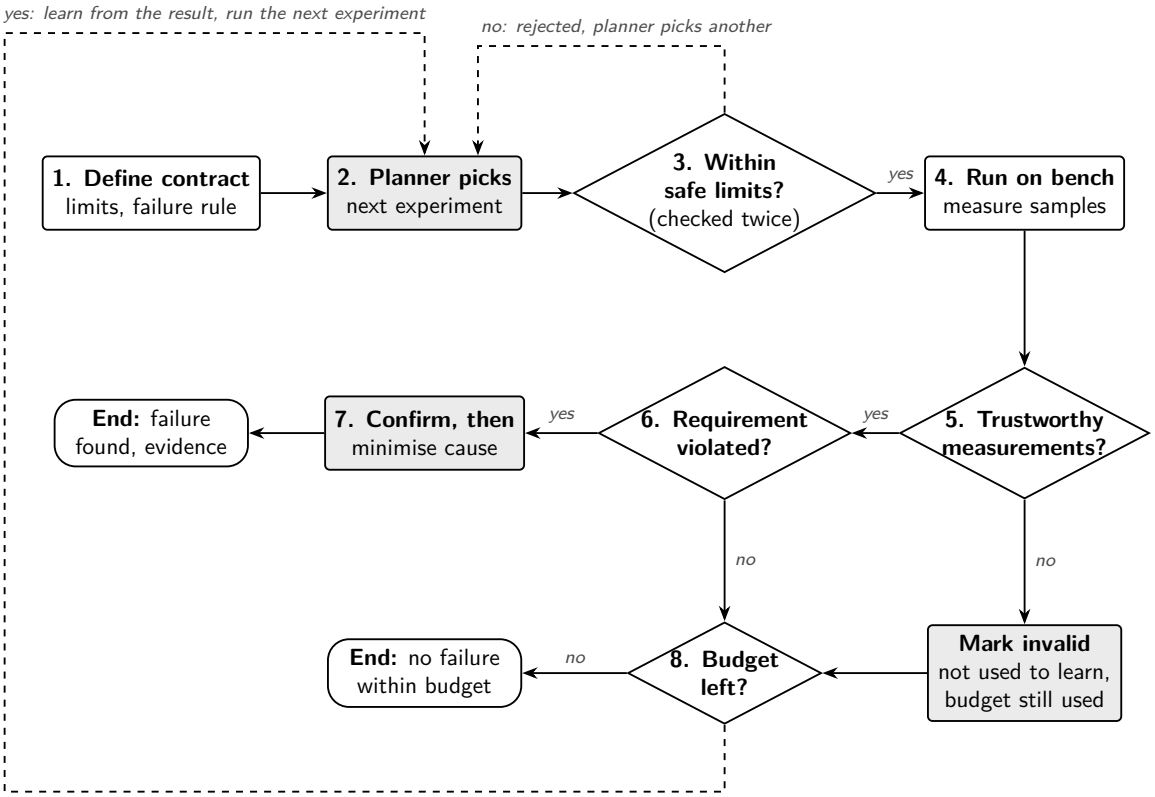


Figure 5.4: The adaptive experiment loop with two safety checks, a trust check on the measurements, confirmation and minimisation.

Table 5.2: Outcome classification (evaluation order)

#	Condition	Outcome	Label	Train
1	Device refused the request or the operator stopped the run	`REJECTED / ABORTED`	none	no
2	Watchdog fired, reset detected or boot identifier changed	`WATCHDOG_RESET`	1	yes
3	Timeout, communication errors or zero samples	`COMM_ERROR`	none	no
4	One or more samples missed the deadline (under 10% transient, otherwise severe)	`TIMING_FAILURE`	1	yes
5	Jitter above the larger of 1.5 ms and 25% of the interval	`TIMING_FAILURE`	1	yes
6	Invalid sensor readings (kept apart from timing failures)	`SENSOR_FAULT`	0	yes
7	None of the above	`PASS`	0	yes

Step 5: signatures, confirmation and minimisation. A failure receives a reproducible signature built from its class, sensor, interval bucket and load bucket, for example

`DEADLINE_MISS:SEVERE:LDR:p10ms:load60`

so equivalent failures are counted once. A failure is repeated to confirm it, then conditions are removed or moved toward nominal while the failure persists, which gives the minimal cause.

Baselines. Random, grid, TPE [21] and space-filling selection use the same loop, device, budget, seed policy and oracle as the proposed planner. A Gaussian-process Bayesian-optimisation planner is kept as an optional comparison.

Novelty versus adopted work. The surrogate models, TPE, delta debugging and the hardware-in-the-loop bench are adopted from prior work. What this project contributes is their combination in one safety-constrained physical loop, the two-layer deterministic safety check, the independent recomputation of metrics from raw samples with a frozen oracle, and a reproducible evaluation on identical budgets.

Simulator and seeded defects. Before hardware is connected, a deterministic ESP32 simulator speaks the identical JSONL frames, re-validates every request like firmware, and seeds its noise from a hash of the noise seed, experiment identifier and configuration. It contains three disclosed timing defects, shown in Table 5.3, that the planner never reads, plus fault-injection switches (forced watchdog reset, forced communication timeout, forced sensor glitch and a corrupted firmware summary) to test the safety and verification paths. Failures found in simulation validate the pipeline; they say nothing about the physical firmware, and simulated and hardware results are never pooled.

Table 5.3: Seeded timing defects in the simulator (disclosed ground truth)

Defect	Trigger region	Effect
A: high-load scheduling contention	interval $\leq$ 15 ms and load $\geq$ 65	Queuing delay and jitter spikes
B: tight deadline with POT ADC stalls	POT sensor, load $\geq$ 50, deadline $\leq$ 10 ms	Stretched execution and lateness
C: watchdog starvation	interval $\leq$ 5 ms and load $\geq$ 90	Watchdog reset, new boot identifier

5.5 Operator Console

The researcher works through the console. It lists campaigns and their state, shows each approved contract with its requirements, observations and safety limits, shows the bench's

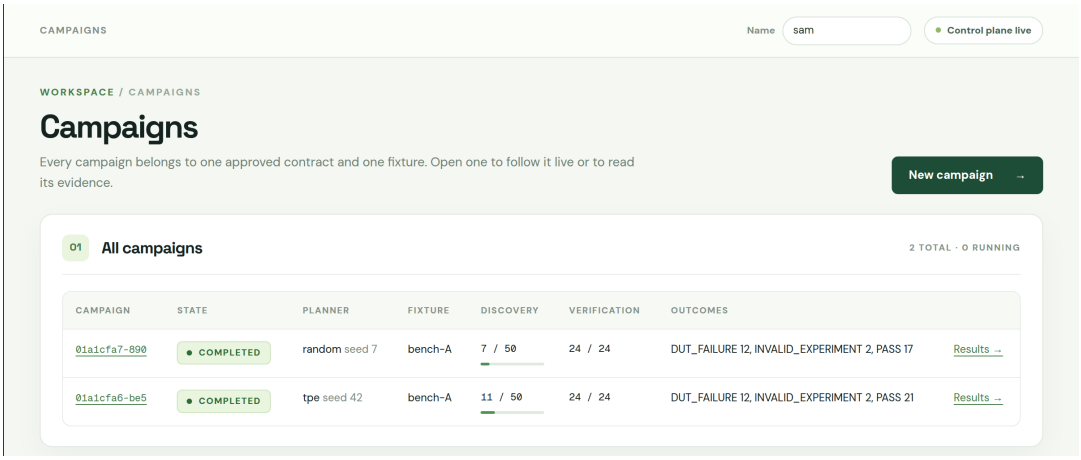


Figure 5.5: Console: campaign list with state, planner, fixture, budget use and outcome counts.

health and safety state, follows a running campaign live with its outcome timeline and event log, and compares planners on the same contract. Figures 5.5 to 5.9 are screenshots of the console running a demonstration campaign of the OLED rig on the simulated bench with dummy data (a TPE campaign and a random campaign on the same contract).

5.6 Implementation Plan (Phase-wise)

Table 5.4: Implementation plan

Phase	Task to be completed	Activities involved	Duration
I	Initiation	Research question, scope freeze (one ESP32 DUT, extra-low-voltage only), literature review, team roles, instrument inventory, Phase-1 demonstration	Weeks 1–2
II	Planning & Design	System, hardware and safety-state-machine design; test contract and safety limits; simulator, planner interface and baseline planners. Gate (week 4): a safe, remotely controllable setup with read-back, or pivot to simulation	Weeks 3–4
III	Safe Instrument Control & Firmware	Test-controller, watchdog, E-stop and calibration; ESP32 firmware and the JSONL protocol; hardware bring-up checklist. Gate (week 8): 100 nominal experiments with under 5% invalid	Weeks 5–8
IV	Adaptive Search Integration	ExtraTrees and baseline planners on the loop, failure confirmation and minimiser, dashboard and exports; weekly live demonstration	Weeks 9–10

V	Safety Acceptance & Evaluation	Safety acceptance suite. Gate (week 12): zero unsafe executions. Repeated seeded campaigns of adaptive versus baseline strategies; analysis	Weeks 11–12
VI	Documentation & Reporting	Freeze code and hardware, archive raw data with checksums, final report and paper draft	Week 13

5.7 Tools & Technologies

Table 5.5: Tools and technologies

Category	Tools / Technologies / Versions
Programming Language	Python 3.12 for all application services; TypeScript for the console; firmware for the ESP32 (toolchain chosen by the hardware team)
Frameworks / Libraries	NumPy, Optuna (TPE and samplers), optional PyTorch for the Gaussian-process planner, Next.js 15, React 19
Development Environment	Linux workstation, `uv` virtual environments, command-line test runner (Python `unittest`) and TypeScript type checking
Database / Storage	SQLite relational tables for devices, runs, experiments, raw samples, summaries and an append-only event log; trace files on disk
Hardware / Platform	Laptop; ESP32-WROOM-32 DevKit; breadboard; LDR and trimmer potentiometer; micro-USB cable. Multi-rail rig: ESP32-S3 device, RP2040 test controller, three programmable rails, E-stop and fuse
Version Control	Git and GitHub
Other Tools	Deterministic ESP32 simulator and fault-injection switches; CSV and JSON export of raw samples; Node.js scripts for the isometric figures

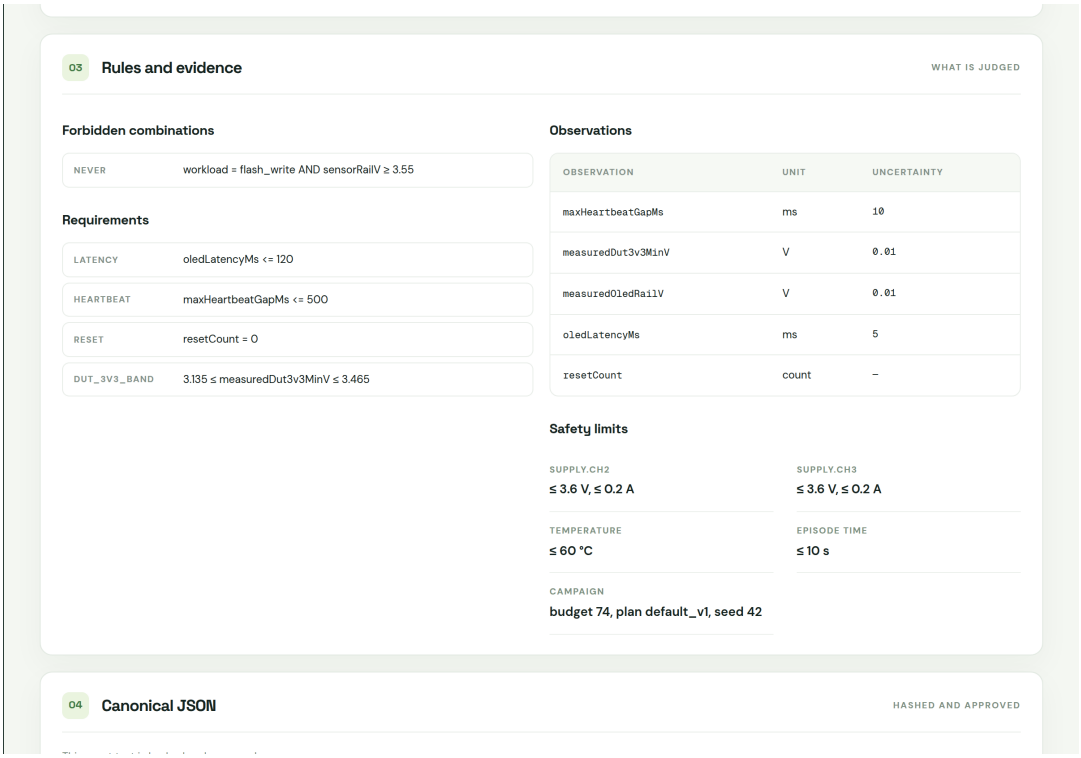


Figure 5.6: Console: an approved contract showing forbidden combinations, requirements, observations with their uncertainties and the hard safety limits.

5.8 Risk Analysis and Mitigation

Table 5.6: Project risks and mitigation

Risk	Mitigation
Real firmware timing differs from the simulator's assumptions	Hardware acceptance gate (checklist steps 1–13) before any hardware result; simulated and hardware results reported separately
Seeded defects make the problem artificial	Disclose them as ground truth hidden from the planner; add hardware-only runs; never pool with simulation
Serial throughput cannot carry 4096 samples at high rates	Buffer on the device, transmit outside the timed window, reduce samples
`task_load` is not reproducible across boards	Per-board calibration table recorded with every campaign
A small search space lets a grid win	Report it honestly; it is a valid result
The adaptive method does not beat the baselines	A valid, documented outcome of the research question

Signature bucket widths change the count of unique failures	Freeze them before the final evaluation and report sensitivity
Unreviewed code reaches the bench	Code review, boundary tests, the verification checklist and the safety acceptance suite

6. Expected Outcomes & Deliverables

Upon successful completion of this project, the following outcomes are expected:

6.1 Technical Outcomes

- A working testbench in which every candidate passes two independent safety checks, with zero rejected configurations executed in all campaigns.
- A JSON-Lines protocol and an ESP32 bring-up that return microsecond raw samples, with the PC recomputation agreeing with the firmware summary within the stated tolerances (0.05 ms for mean interval and jitter, 0.002 for miss rate) on a clean run, and a corrupted summary always detected.
- An adaptive planner, three ablations and four baselines on one loop, with identical output for identical seeds in simulation.
- A console with seven views (device status, experiment configuration, live monitor, results and raw telemetry, adaptive testing, research comparison and audit log) and CSV and JSON export of raw samples.

6.2 Research Outcomes

- Evidence, with confidence intervals over repeated seeds, on whether adaptive selection reaches the first timing failure and finds more distinct failure signatures than random and grid selection at equal budgets of 15, 30 and 50 experiments. The hypothesis that the median experiments-to-first-failure is at least 30% lower than random is to be tested and may be rejected.
- An ablation showing which term of the acquisition matters, and a measured discrepancy rate for firmware self-reports.
- A reproducible open design and a paper draft, kept honest about simulation versus hardware and about what the absence of failures does not prove.

6.3 Deliverables Checklist

Table 6.1: Deliverables and target timeline

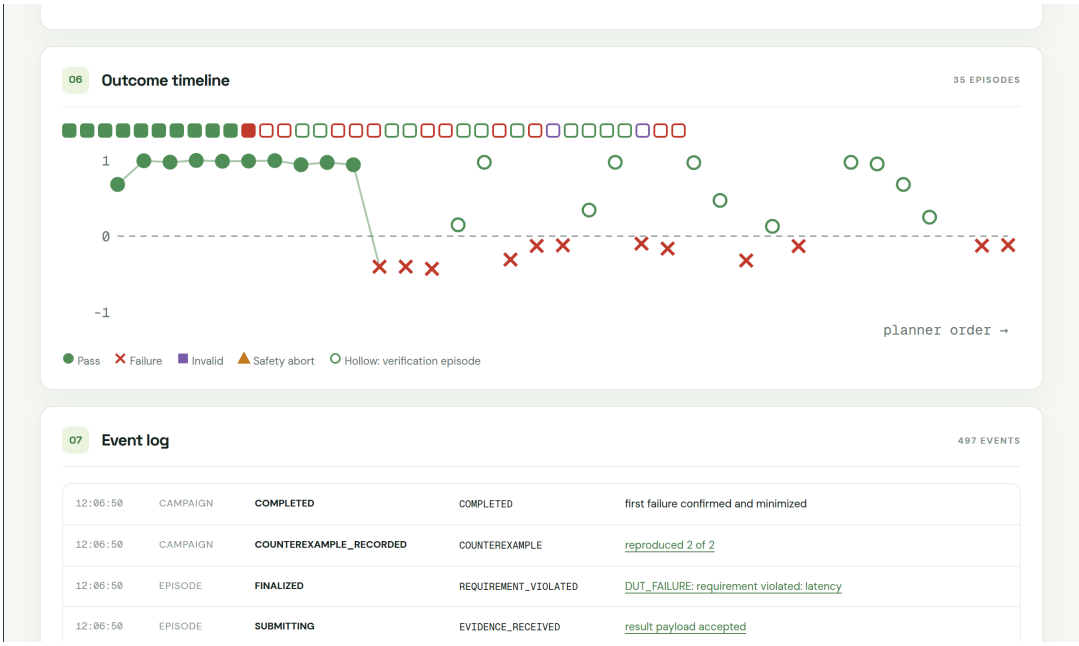


Figure 5.7: Console: the outcome timeline of a finished campaign (passes, failures, invalid episodes, and hollow marks for verification episodes) above the event log.

Deliverable	**Target Timeline**
Synopsis / Proposal Document	Week 2
Literature Review Report	Week 2
Simulator, Planner Interface and Baseline Planners	Week 4
Safe Bench with ESP32 Firmware and Bring-up Evidence	Week 8
Integrated Adaptive Campaigns, Console and Code Repository	Week 10
Evaluation Report & Results	Week 12
Final Project Report, Paper Draft & Presentation	Week 13

7. Evaluation Metrics & Validation Plan

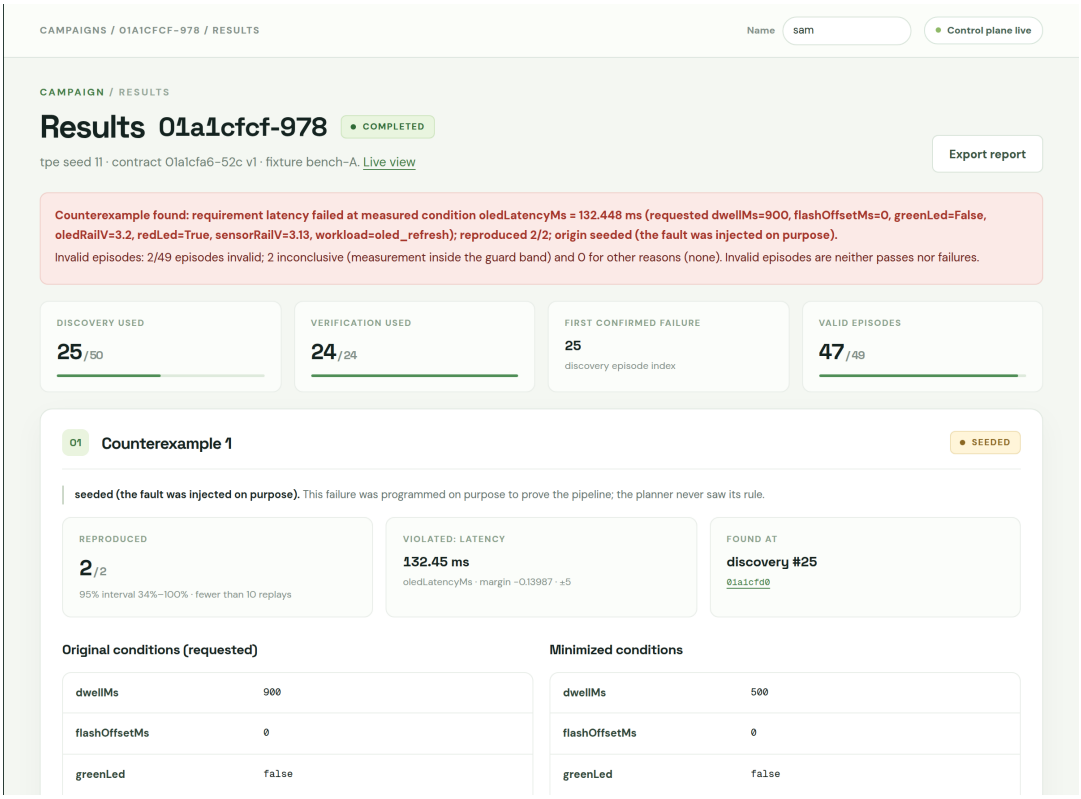


Figure 5.8: Console: results of a finished campaign. A counterexample is reported with its reproduction count, the violated requirement and the original and minimised conditions, and a failure injected on purpose is labelled as seeded.

Research questions.

Table 7.1: Research questions and hypotheses (to be pre-registered before the final runs)

ID	Question	Hypothesis
RQ-T1	Does the full adaptive strategy reach the first timing failure in fewer experiments than random and grid at equal budget?	Median experiments-to-first-failure is at least 30% lower than random (may be rejected)
RQ-T2	Does it find more distinct failure signatures per budget?	Higher unique-signature count at budgets 15, 30 and 50
RQ-T3	Which acquisition term matters?	Removing failure probability hurts most; removing novelty lowers diversity
RQ-T4	Are firmware self-reports trustworthy?	Discrepancy rate near zero on healthy hardware; injected corruption always detected
RQ-T5	Do simulator conclusions transfer to hardware?	Exploratory; reported as such

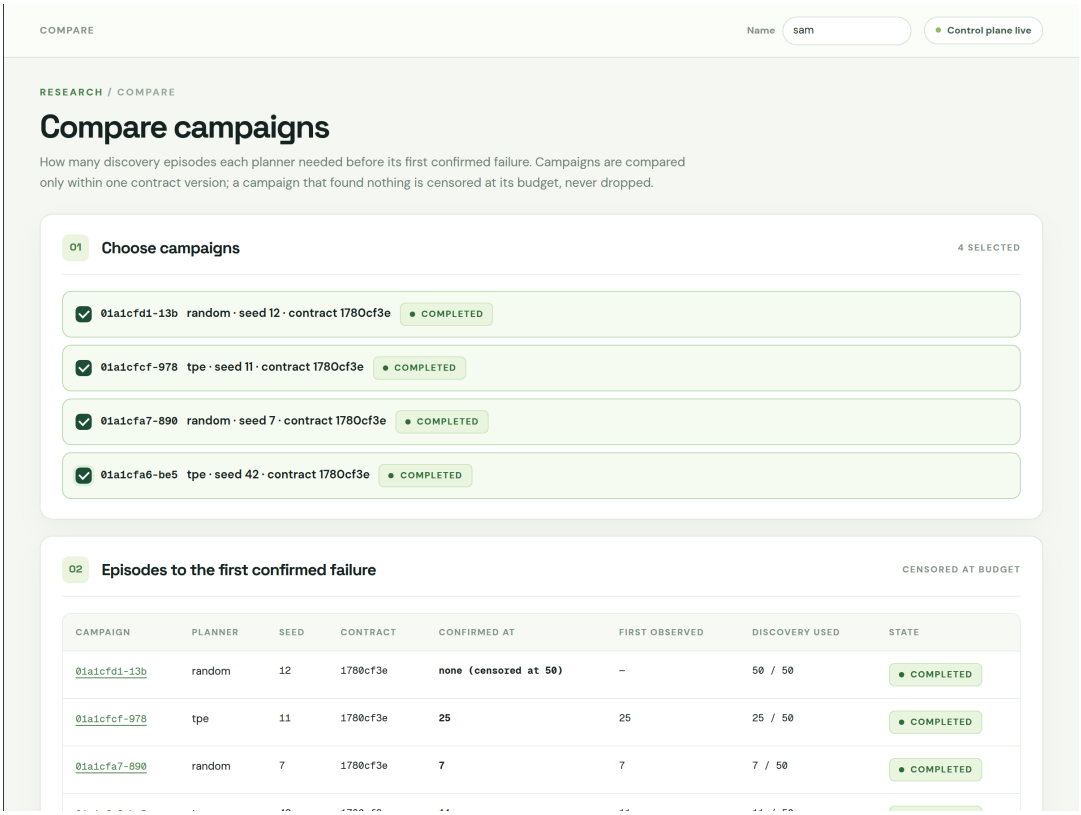


Figure 5.9: Console: comparison of planners on one contract. Episodes to the first confirmed failure are right-censored at the budget, so a campaign that found nothing in 50 episodes is shown as censored and not dropped.

Metrics:

- *Primary:* discovery rate at budget; experiments to the first failure, right-censored at the budget; number of unique failure signatures; discovery curve (unique failures against experiments); time to first failure measured as cumulative experiment time and not as wall-clock time.
- *Secondary:* mean deadline-miss rate and jitter of executed experiments, count of rejected configurations, discrepancy count, planner compute time.
- *Safety and integrity:* number of unsafe or rejected configurations executed (target exactly zero), discrepancy detection of injected corruption, and symmetric rejection by both the PC and the device.

Validation strategy:

- Strategies: the full planner, three ablations, random, grid, TPE and space-filling selection, all on the same device, fault configuration, noise-seed policy and oracle, with the order of strategies randomised on hardware.
- Budgets of 15, 30 and 50 experiments, with at least 30 independent seeds per strategy and budget where time allows.

- Medians and interquartile ranges with 95% bootstrap confidence intervals, Kaplan–Meier curves for censored time to failure, and paired comparison on matched seeds with effect sizes. The primary metric is stated before the runs, oracle thresholds, tolerances and signature bucket widths are frozen before the final evaluation, all seeds and raw logs are published, and there is no tuning after the results are seen.
- Simulation and hardware results are reported separately and labelled as such, and real-hardware evaluation starts only after the bring-up checklist (steps 1 to 13) passes.

Threats to validity. The seeded defects are artificial, and a small search space can favour a grid; both are disclosed and reported honestly. Calibrated work units may differ between boards, so each board is calibrated and the calibration is recorded. Timestamps come from the ESP32 clock and never from the host, so USB scheduling noise on the laptop does not enter any metric. Thermal and state drift on hardware is checked with a reference experiment between blocks and by logging boot identifiers and reset reasons.

Ethical and safety considerations.

- No human subjects or personal data are involved.
- Version 1 adds no electrical hazard beyond USB-powered bench work, and the multi-rail rig keeps every supply within a reviewed extra-low-voltage envelope with a hardware cut-off.
- Results use honest language: a counterexample was found, or no failure was found within the budget. Neither is ever described as proof that the device is safe.

8. References

[1] United Nations Department of Economic and Social Affairs, “Goal 9: Industry, innovation and infrastructure,” targets 9.1–9.c. [Online]. Available: https://sdgs.un.org/goals/goal9

[2] M. Zalewski, “Technical whitepaper for afl-fuzz,” AFL technical documentation. [Online]. Available: https://github.com/google/AFL/blob/master/docs/technical_details.txt

[3] J. Yun, F. Rustamov, J. Kim, and Y. Shin, “Fuzzing of embedded systems: A survey,” *ACM Comput. Surv.*, vol. 55, no. 7, 2023. doi: 10.1145/3538644

[4] M. Eisele, M. Maugeri, R. Shriwas, C. Huth, and G. Bella, “Embedded fuzzing: A review of challenges, tools, and solutions,” *Cybersecurity*, vol. 5, art. 18, 2022. doi: 10.1186/s42400-022-00123-y

[5] A. A. Clements *et al.*, “HALucinator: Firmware re-hosting through abstraction layer emulation,” in *Proc. USENIX Security Symp.*, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/clements

[6] B. Feng, A. Mera, and L. Lu, “P^{2}IM: Scalable and hardware-independent firmware test-

ing via automatic peripheral interface modeling," in *Proc. USENIX Security Symp.*, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/feng

[7] T. Scharnowski *et al.*, "Fuzzware: Using precise MMIO modeling for effective firmware fuzzing," in *Proc. USENIX Security Symp.*, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/scharnowski

[8] W. Li *et al.*, "μAFL: Non-intrusive feedback-driven fuzzing for microcontroller firmware," in *Proc. Int. Conf. Software Engineering (ICSE)*, 2022. doi: 10.1145/3510003.3510208

[9] D. Rusconi *et al.*, "E-FuzzEdge: Efficient in-place firmware fuzzing via parallel scheduling," in *NDSS Workshop on Fuzzing*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/fuzzing2026-4.pdf

[10] D. R. Kuhn, R. N. Kacker, and Y. Lei, "Practical combinatorial testing," NIST Special Publication 800-142, 2010. doi: 10.6028/NIST.SP.800-142

[11] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, 1979. doi: 10.1080/00401706.1979.10489755

[12] B. Bryan, J. Schneider, R. C. Nichol, C. J. Miller, C. R. Genovese, and L. Wasserman, "Active learning for identifying function threshold boundaries," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2005.

[13] A. Gotovos, N. Casati, G. Hitz, and A. Krause, "Active learning for level set estimation," in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2013.

[14] Y. Sui, A. Gotovos, J. Burdick, and A. Krause, "Safe exploration for optimization with Gaussian processes," in *Proc. Int. Conf. Machine Learning (ICML)*, vol. 37, 2015, pp. 997–1005. [Online]. Available: https://proceedings.mlr.press/v37/sui15.html

[15] F. Berkenkamp, A. Krause, and A. P. Schoellig, "Bayesian optimization with safety constraints: Safe and automatic parameter tuning in robotics," *Machine Learning*, vol. 112, pp. 3713–3747, 2023. doi: 10.1007/s10994-021-06019-1

[16] Y. Annapureddy, C. Liu, G. Fainekos, and S. Sankaranarayanan, "S-TaLiRo: A tool for temporal logic falsification for hybrid systems," in *Proc. TACAS*, 2011.

[17] M. Koren, S. Alsaif, R. Lee, and M. J. Kochenderfer, "Adaptive stress testing for autonomous vehicles," in *Proc. IEEE Intelligent Vehicles Symp.*, 2018. [Online]. Available: https://arxiv.org/abs/1902.01909

[18] K. Zandberg *et al.*, "PHiLIP on the HiL: Automated multi-platform OS testing with external reference devices," *ACM Trans. Embedded Comput. Syst.*, vol. 20, no. 5s, 2021. [Online]. Available: https://arxiv.org/abs/2107.07255

[19] IEEE Standards Association, *IEEE 2004-2025: Recommended Practice for Hardware-in-the-Loop Simulation-Based Testing of Electric Power Apparatus and Controls*. [Online]. Available: https://standards.ieee.org/ieee/2004/11300/

[20] T. Hossain, A. Mahadevan, J. Van Woudenberg, R. Velegalati, and A. Bhattacharyya, "GlitchLab: A hardware-in-the-loop optimizer for physical fault injection," arXiv preprint, 1 Sep. 2026 (not peer reviewed). [Online]. Available: https://arxiv.org/

abs/2609.00502

[21] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 24, 2011.

[22] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006. doi: 10.1007/s10994-006-6226-1

[23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019. doi: 10.1145/3292500.3330701

[24] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, 2002. doi: 10.1109/32.988498

[25] Google, "FuzzBench: Fuzzer benchmarking as a service." [Online]. Available: https://github.com/google/fuzzbench

[26] IEC, *IEC 61010-1:2010+AMD1:2016: Safety requirements for electrical equipment for measurement, control, and laboratory use – Part 1: General requirements*. [Online]. Available: https://webstore.iec.ch/en/publication/4279

[27] Espressif Systems, *ESP32-WROOM-32 Datasheet*. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

9. Declaration & Signatures

I/We hereby declare that this project synopsis is our original work carried out as part of the Experiential Learning course. This work has not been submitted elsewhere for any other degree or examination. All sources of information have been duly acknowledged.

Student Signature:	**Guide Signature:**
_____	_____
Kavin Krishnan C (1RV24CS126), Chinmai Kiran Shetti (1RV24CS068), Sumukha Upadhyaya (1RV24IS130), Snehal Reddy Thadigotla (1RV24IS126), Sutej R Haritsa (1RV24CY055)	*Dr. H K Krishappa*
Date: _____	*Date:* _____